

Aula 14 – 19/05/2026

Persistência, JDBC e Controle de
Acesso

Sumário

- Introdução à Persistência
 - O problema da memória volátil
 - O que é SQL?
- JDBC (Java Database Connectivity)
 - Arquitetura de Drivers
 - Interfaces e Polimorfismo no acesso a dados
- O Banco de Dados SQLite
 - Por que usar?
 - Configuração do ambiente
- Prática: Tabela de Usuários
 - Controle de Acesso
 - Via HttpSession (Stateful)
 - Via Tokens (Stateless)

O Problema da Persistência

- **Memória Volátil (RAM):** Até agora, todos os nossos objetos (`ArrayList`, `HashMap`, variáveis) viviam apenas enquanto o programa estava rodando.
- **Reiniciou, perdeu:** Se pararmos o servidor Javalin ou fecharmos o programa, todos os dados (usuários, votos, produtos) somem.
- **A Solução:** Precisamos gravar esses dados em um meio **persistente** (Disco Rígido / SSD).
- **Banco de Dados Relacional (RDBMS):** É a forma mais tradicional e robusta de estruturar e armazenar dados corporativos.

Introdução ao SQL e JDBC



SQL (Structured Query Language): É a linguagem universal para "conversar" com bancos de dados.

- Comandos principais de DML (Data Manipulation Language):
`CREATE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`.

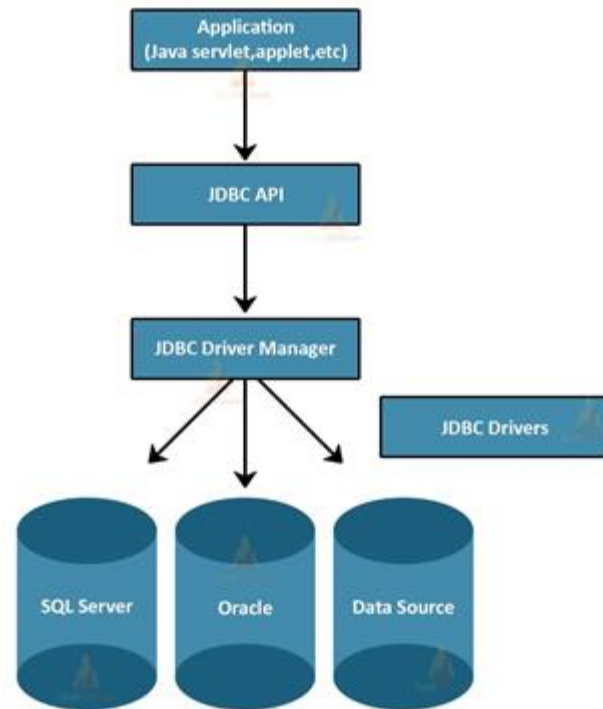
JDBC (Java Database Connectivity):

- O Java não fala "nativamente" com o banco de dados.
- O JDBC é uma **Especificação** (um conjunto de **Interfaces** Java) que define *como* uma aplicação deve interagir com um banco.
- Lembra da aula de Interfaces? O JDBC diz: "Todo banco deve ter um método `connect()`, um `execute()`, etc."

A Arquitetura JDBC e os Drivers

- Se o JDBC é apenas um conjunto de interfaces (o contrato), quem faz o trabalho pesado?
- **O Driver:** É uma biblioteca (arquivo `.jar`) criada pelo fabricante do banco de dados (Oracle, Microsoft, PostgreSQL, SQLite) que **implementa** as interfaces do JDBC.
- **Fluxo:**
 1. Sua Aplicação Java chama métodos do JDBC (padrão).
 2. O JDBC repassa para o **Driver** específico (ex: Driver do SQLite).
 3. O Driver traduz para o protocolo binário do banco de dados.

Vantagem: Seu código Java quase não muda se você trocar de banco de dados. Você programa para a **Interface**, não para a implementação!



SQLite



Para nossas aulas, não vamos instalar um servidor pesado (como MySQL ou Oracle). Vamos usar o **SQLite**.

O que é:

- Um banco de dados **serverless** (não precisa instalar/rodar um serviço em background).
- Todo o banco de dados é salvo em um **único arquivo** no seu projeto.
- Zero configuração.

Por que SQLite?

- Extremamente leve e rápido.
- O processo de uso do JDBC com ele é **idêntico** ao uso com bancos gigantes corporativos.
- Ideal para prototipagem e aprendizado.

Padronização do Código

Graças ao polimorfismo, o código para conectar é padrão. O que muda é apenas a **Connection String** (URL de conexão).

- **Conectando no SQLite:** `String url = "jdbc:sqlite:meu_banco.db";`
- **Conectando no MySQL (exemplo):** `String url = "jdbc:mysql://localhost:3306/meu_banco";`
- **Conectando no PostgreSQL (exemplo):** `String url = "jdbc:postgresql://localhost:5432/meu_banco";`

O resto do código Java (`Connection`, `Statement`, `ResultSet`) permanece o mesmo!

Dependência Maven

<https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc>

```
<dependency>  
    <groupId>org.xerial</groupId>  
    <artifactId>sqlite-jdbc</artifactId>  
    <version>3.51.0.0</version>  
</dependency>
```


Criando um projeto

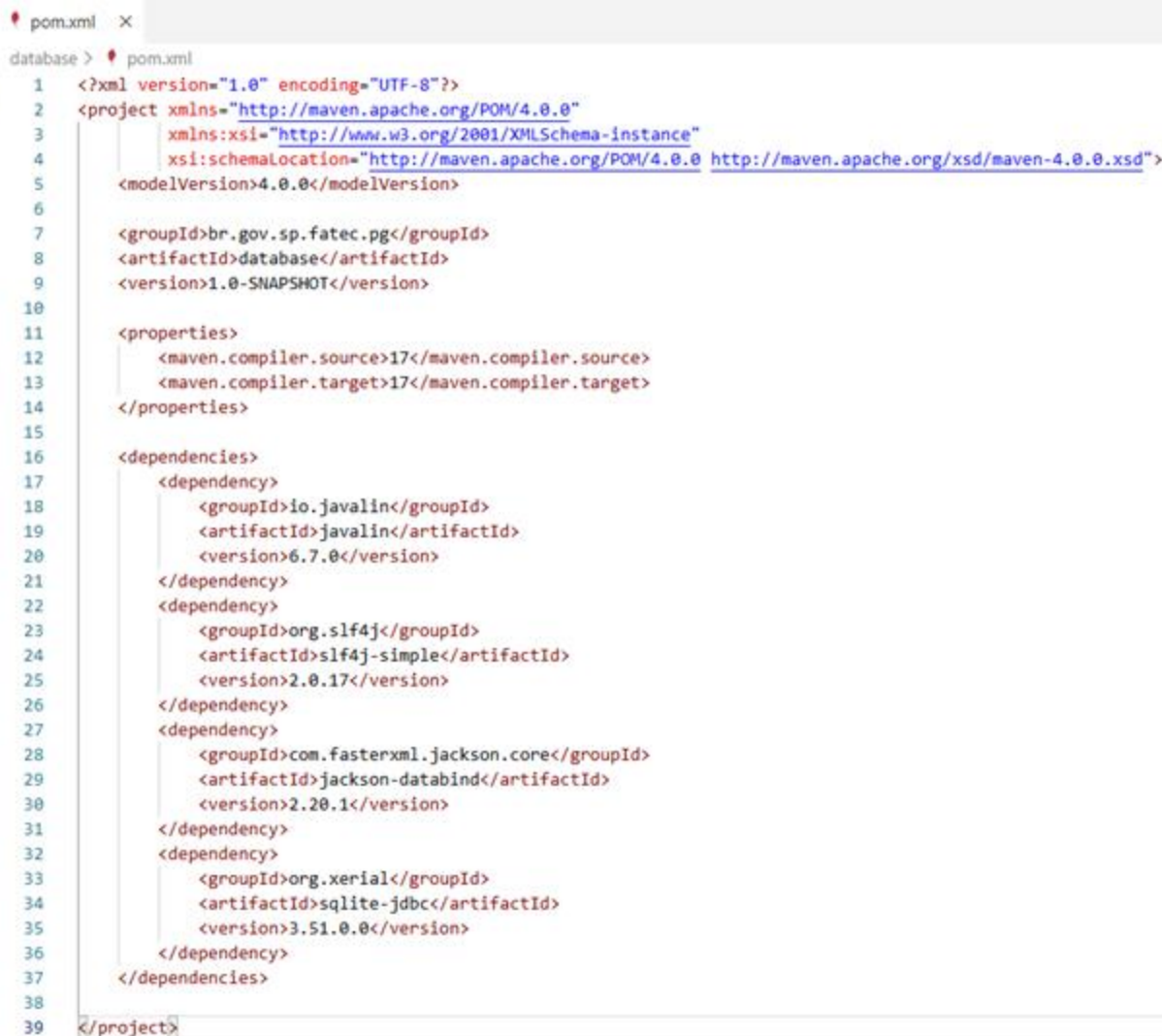
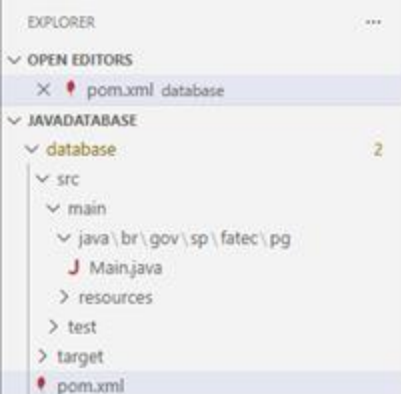
Vamos criar a estrutura básica. Se você já tem o projeto Javalin da aula anterior, pode aproveitar, mas certifique-se de que a dependência do **SQLite JDBC** está no `pom.xml`.

Estrutura de Pastas Sugerida:

- `src/main/java`
 - `br.gov.sp.fatec.pg`
 - `database` (Para conexão e tabelas)
 - `model` (Para a classe User)
 - `repository` (Para as operações SQL)
 - `Main.java` (Onde fica o Javalin)

Incluindo as dependências no pom.xml

- As mesmas usadas na aula passada:
 - javalin
 - sl4j-simple
 - jackson-databind
- O driver do SQLite
 - sqlite-jdbc



Implementando conexão e criação de tabela

1. Classe de Conexão e criação da tabela users (**SQLiteConnection.java**)

Caminho: `src/main/java/br/gov/sp/fatec/pg/database/SQLiteConnection.java`

2. Criação do servidor Javalin (**Main.java**)

Caminho: `src/main/java/br/gov/sp/fatec/pg/Main.java`

OPEN EDITORS

X J SQLiteConnection.java database\src\mai...

JAVADATABASE

database 2

src

main

java\br\gov\sp\fathec\pg

database

J SQLiteConnection.java

J Main.java

> resources

> test

> target

pom.xml

database > src > main > java > br > gov > sp > fathec > pg > database > J SQLiteConnection.java > SQLiteConnection

```
1 package br.gov.sp.fatec.pg.database;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.Statement;
6
7 public class SQLiteConnection {
8     // String de conexão (cria o arquivo users.db na raiz do projeto)
9     private static final String URL = "jdbc:sqlite:users.db";
10
11     public static Connection connect() throws Exception {
12         return DriverManager.getConnection(URL);
13     }
14
15     // Método para criar a tabela se ela não existir
16     public static void createTable() {
17         String sql = "CREATE TABLE IF NOT EXISTS users (" +
18             "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
19             "username TEXT UNIQUE NOT NULL, " +
20             "password TEXT NOT NULL)";
21
22         // Try-with-resources garante o fechamento da conexão
23         try (Connection conn = connect();
24             Statement stmt = conn.createStatement()) {
25             stmt.execute(sql);
26             System.out.println("Tabela 'users' verificada/criada com sucesso.");
27         } catch (Exception e) {
28             e.printStackTrace();
29         }
30     }
31 }
```

OPEN EDITORS

J SQLiteConnection.java database\src\mai...

X J Main.java database\src\main\java\br\gov\...

JAVADATABASE

database 2

src

main

java\br\gov\sp\fatcc\pg

database

J SQLiteConnection.java

J Main.java

> resources

> test

> target

pom.xml

users.db

database > src > main > java > br > gov > sp > fatec > pg > J Main.java > Main > main(Stri

```
1 package br.gov.sp.fatec.pg;
2
3 import br.gov.sp.fatec.pg.database.SQLiteConnection;
4 import io.javalin.Javalin;
5
6 public class Main {
7     Run | Debug
8     public static void main(String[] args) {
9         // 1. Garante que a tabela existe ao iniciar
10        SQLiteConnection.createTable();
11
12        Javalin app = Javalin.create().start(7070);
13
14        // 2. Rota de Teste (Logger)
15        app.get("/logger", ctx -> {
16            // Se chegou aqui sem erros no console, o banco está ok
17            ctx.result("Banco de dados conectado e tabela pronta!");
18        });
19    }
```

Testando

```
9 public class Main {  
    Run | Debug  
10 public static void main(String[] args) {  
11     // 1. Garante que a tabela existe ao iniciar  
12     SQLiteConnection.createTable();  
}
```

OUTPUT DEBUG CONSOLE PORTS

▼ TERMINAL Run: Main

PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaDatabase> & 'C:\Program Files\Java\openjdk\bin\java.exe' '@C:\Users\rlarg\AppData\Local\Temp\cp_93dvazg3ih4cp8cr8j03hvvr.d\argfile' sp.fatec.pg.Main

Tabela 'users' verificada/criada com sucesso.

localhost:7070/logger

localhost:7070/logger

Banco de dados conectado e tabela pronta!

O Modelo e o Repositório

1. Classe de Modelo (**User.java**)

Esta classe é um POJO (Plain Old Java Object). É essencial ter o **construtor vazio** para que o Javalin consiga converter o JSON recebido diretamente para este objeto.

Caminho: `src/main/java/br/gov/sp/fatec/pg/model/User.java`

2. Classe de Repositório (**UserRepository.java**)

Esta classe abstrai toda a complexidade do JDBC. Observe o uso de `try-with-resources` (os parênteses logo após o `try`) para garantir que a conexão com o SQLite seja fechada automaticamente, evitando vazamento de memória ou travamento do arquivo do banco.

Caminho: `src/main/java/br/gov/sp/fatec/pg/repository/UserRepository.java`

OPEN EDITORS

X J User.java database\src\main\java\br\gov\fs...

JAVADATABASE

database 2

src

main

java\br\gov\sp\fatcc\pg

database

J SQLiteConnection.java

model

J User.java

J Main.java

> resources

> test

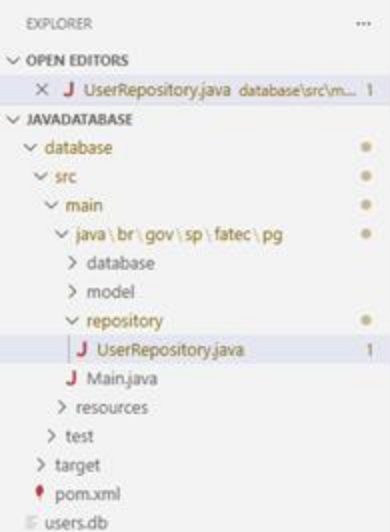
> target

pom.xml

users.db

database > src > main > java > br > gov > sp > fatcc > pg > model > J User.java > User

```
1 package br.gov.sp.fatec.pg.model;
2
3 /* Modelo de usuário para representar os dados do usuário. Padrão DAO/Repository.*/
4 public class User {
5
6     private Integer id;
7     private String username;
8     private String password;
9
10    public User() {}
11
12    public User(String username, String password) {
13        this.username = username;
14        this.password = password;
15    }
16
17    public Integer getId() {return id;}
18
19    public void setId(Integer id) {this.id = id;}
20
21    public String getUsername() {return username;}
22
23    public void setUsername(String username) {this.username = username;}
24
25    public String getPassword() {return password;}
26
27    public void setPassword(String password) {this.password = password;}
28
29    @Override
30    public String toString() {
31        return "User{username='" + username + "'}"; // Nunca inclua a senha no toString!
32    }
33 }
```



```
database > src > main > java > br > gov > sp > fatec > pg > repository > UserRepository.java > UserRepository

1  package br.gov.sp.fatec.pg.repository;
2
3  import br.gov.sp.fatec.pg.database.SQLiteConnection;
4  import br.gov.sp.fatec.pg.model.User;
5
6  import java.sql.Connection;
7  import java.sql.PreparedStatement;
8  import java.sql.ResultSet;
9  import java.sql.SQLException;
10
11 public class UserRepository {
12
13     /**
14      * Insere um novo usuário no banco de dados.
15      * @param user Objeto usuário preenchido
16      * @throws Exception Se houver erro de SQL ou duplicação de username
17      */
18     public static void add(User user) throws Exception {
19         String sql = "INSERT INTO users(username, password) VALUES(?, ?)";
20
21         try (Connection conn = SQLiteConnection.connect();
22             PreparedStatement pstmt = conn.prepareStatement(sql)) {
23
24             pstmt.setString(parameterIndex: 1, user.getUsername());
25             pstmt.setString(parameterIndex: 2, user.getPassword());
26
27             pstmt.executeUpdate();
28         }
29         // A conexão fecha sozinha aqui graças ao try-with-resources
30     }
31
32     /** ...
33     public static boolean delete(String username) throws Exception { ...
34
35     /** ...
36     public static boolean validate(String username, String password) throws Exception { ...
37
38     /** ...
39     public static User getByUsername(String username) throws Exception { ...
40 }
```

OPEN EDITORS

X J UserRepository.java database/src/m... 1

JAVADATABASE

database

src

main

java\br\gov\sp\fatcc\pg

> database

> model

repository

J UserRepository.java 1

J Main.java

> resources

> test

> target

pom.xml

users.db

database > src > main > java > br > gov > sp > fatcc > pg > repository > J UserRepository.java > UserRepository > delete(String)

```
11 public class UserRepository {
12
13 > /** ...
18 > public static void add(User user) throws Exception { ...
31
32 /**
33  * Remove um usuário pelo nome de usuário.
34  * @param username Nome do usuário a ser removido
35  * @return true se apagou, false se não encontrou
36  * @throws Exception Erros de conexão
37  */
38 public static boolean delete(String username) throws Exception {
39     String sql = "DELETE FROM users WHERE username = ?";
40
41     try (Connection conn = SQLiteConnection.connect();
42         PreparedStatement pstmt = conn.prepareStatement(sql)) {
43
44         pstmt.setString(parameterIndex: 1, username);
45
46         int affectedRows = pstmt.executeUpdate();
47         return affectedRows > 0;
48     }
49
50
51 > /** ...
58 > public static boolean validate(String username, String password) throws Exception { ...
75
76 > /** ...
79 > public static User getByUsername(String username) throws Exception { ...
98 }
```

EXPLORER

OPEN EDITORS

- UserRepository.java database\src\m...

JAVADATABASE

- database
 - src
 - main
 - java\br\gov\sp\fatcc\pg
 - database
 - model
 - repository
 - UserRepository.java
 - Main.java
 - resources
 - test
 - target
 - pom.xml
 - users.db

UserRepository.java 1 X

database > src > main > java > br > gov > sp > fatcc > pg > repository > UserRepository.java > UserRepository > validate(String, String)

```
11 public class UserRepository {
12
13 > /**...
18 > public static void add(User user) throws Exception { ...
31
32 > /**...
38 > public static boolean delete(String username) throws Exception { ...
50
51 /**
52  * Valida se o par username/senha existe no banco.
53  * @param username Nome do usuário
54  * @param password Senha (em texto plano para este exemplo didático)
55  * @return true se as credenciais forem válidas
56  * @throws Exception Erros de conexão
57  */
58 public static boolean validate(String username, String password) throws Exception {
59     String sql = "SELECT count(*) FROM users WHERE username = ? AND password = ?";
60
61     try (Connection conn = SQLiteConnection.connect();
62          PreparedStatement pstmt = conn.prepareStatement(sql)) {
63
64         pstmt.setString(parameterIndex: 1, username);
65         pstmt.setString(parameterIndex: 2, password);
66
67         ResultSet rs = pstmt.executeQuery();
68         if (rs.next()) {
69             // Se o count for maior que 0, o usuário existe
70             return rs.getInt(columnIndex: 1) > 0;
71         }
72     }
73     return false;
74
75
76 > /**...
79 > public static User getByUsername(String username) throws Exception { ...
98 }
```

EXPLORER

OPEN EDITORS

- UserRepository.java database/src/m... 1

JAVADATABASE

- database
 - src
 - main
 - java\br\gov\sp\fathec\pg
 - database
 - model
 - repository
 - UserRepository.java 1
 - Main.java
 - resources
 - test
 - target
 - pom.xml
 - users.db

UserRepository.java 1 X

database > src > main > java > br > gov > sp > fathec > pg > repository > UserRepository.java > UserRepository > getUsername(String)

```
11 public class UserRepository {
12
13 > /**...
18 > public static void add(User user) throws Exception { ...
31
32 > /**...
38 > public static boolean delete(String username) throws Exception { ...
50
51 > /**...
58 > public static boolean validate(String username, String password) throws Exception { ...
75
76 /**
77  * (Opcional) Busca um usuário pelo nome. Útil para Sessão.
78 */
79 public static User getUsername(String username) throws Exception {
80     String sql = "SELECT * FROM users WHERE username = ?";
81     User user = null;
82
83     try (Connection conn = SQLiteConnection.connect();
84         PreparedStatement pstmt = conn.prepareStatement(sql)) {
85
86         pstmt.setString(parameterIndex: 1, username);
87         ResultSet rs = pstmt.executeQuery();
88
89         if (rs.next()) {
90             user = new User();
91             user.setId(rs.getInt(columnLabel: "id"));
92             user.setUsername(rs.getString(columnLabel: "username"));
93             user.setPassword(rs.getString(columnLabel: "password"));
94         }
95     }
96     return user;
97
98 }
```

Rotas da API

De volta ao `Main.java`, vamos expor esses métodos via HTTP.

- `/users`
- `/users/{username}`
- `/login`

EXPLORER

OPEN EDITORS 1 unsaved

- UserRepository.java database/src/main/...
- Main.java database/src/main/java/br/gov/...

JAVADATABASE

- database
 - src
 - main
 - java \br\gov\sp\fatcc\pg
 - database
 - model
 - repository
 - UserRepository.java
 - Main.java
- resources
 - test
 - target
 - pom.xml
 - users.db

```

J UserRepository.java 1
J Main.java
database > src > main > java > br > gov > sp > fatcc > pg > J Main.java > Main > main(String[])
6 import br.gov.sp.fatcc.pg.repository.UserRepository;
7 import br.gov.sp.fatcc.pg.model.User;
8
9 public class Main {
    Run | Debug
10     public static void main(String[] args) {
11         // 1. Garante que a tabela existe ao iniciar
12         SQLiteConnection.createTable();
13
14         Javalin app = Javalin.create().start(7070);
15
16         // 2. Rota de Teste (Logger)
17         app.get("/logger", ctx -> { ...
21
22         // 3. Cadastrar Usuário (POST)
23         app.post("/users", ctx -> {
24             User user = ctx.bodyAsClass(User.class); // Transforma JSON em Objeto Java
25             try {
26                 UserRepository.add(user);
27                 ctx.status(201).result("Usuário criado!");
28             } catch (Exception e) {
29                 ctx.status(400).result("Erro ao criar: " + e.getMessage());
30             }
31         });
32
33         // 4. Deletar Usuário (DELETE)
34         app.delete("/users/{username}", ctx -> {
35             String username = ctx.pathParam("username");
36             if (UserRepository.delete(username)) {
37                 ctx.status(204); // Sucesso, sem conteúdo
38             } else {
39                 ctx.status(404).result("Usuário não encontrado.");
40             }
41         });
42
43         // 5. Login (POST) - Simples validação
44         app.post("/login", ctx -> {
45             User user = ctx.bodyAsClass(User.class);
46             if (UserRepository.validate(user.getUsername(), user.getPassword())) {
47                 ctx.status(200).result("Login com sucesso!");
48             } else {
49                 ctx.status(401).result("Credenciais inválidas.");
50             }
51         });
52
53     }

```

Roteiro de teste via Postman (desktop)

1. Verificar Saúde:

- a. **GET** `http://localhost:7070/logger`
- b. *Esperado:* 200 OK "Banco de dados conectado..."

2. Criar Usuário:

- a. **POST** `http://localhost:7070/users`
- b. **Body (JSON):** `{ "username": "admin", "password": "123" }`
- c. *Esperado:* 201 Created.

3. Testar Login (Sucesso):

- a. **POST** `http://localhost:7070/login`
- b. **Body (JSON):** `{ "username": "admin", "password": "123" }`
- c. *Esperado:* 200 OK.

4. Testar Login (Falha):

- a. Mude a senha para "errada".
- b. *Esperado:* 401 Unauthorized.

5. Deletar Usuário:

- a. **DELETE** `http://localhost:7070/users/admin`
- b. *Esperado:* 204 No Content.

Roteiro de teste via Console do Chrome

1. Certifique-se de que seu servidor Java (Main.java) esteja rodando no VS Code.
2. Abra o Google Chrome.
3. Pressione **F12** (ou clique com o botão direito -> Inspecionar).
4. Clique na aba **Console**.
5. **Copie e cole** os comandos a seguir, um por um, e pressione **Enter**.

Banco de dados conectado e tabela pronta!

Elements Console Source Network Performance Memory Application Privacy and security Lighthouse Recorder

top -url:https://www.gstatic.com/_/js/boq-chrome-webstore/_/js/boq-chrome-webstore.ChromeWebStoreConsumerFeiUigt_8RtuHnaEX7ib Custom levels No Issues

8 messages
6 user messages
2 errors
No warnings
6 info
No verbose

```
fetch('http://localhost:7070/logger')
  .then(response => response.text())
  .then(texto => console.log("%c STATUS DO SERVIDOR: " + texto, "color: green; font-weight: bold;"));
< Promise {<pending>}
STATUS DO SERVIDOR: Banco de dados conectado e tabela pronta! VM72:3
fetch('http://localhost:7070/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: "aluno_fatec",
    password: "segredo"
  })
})
.then(res => res.text()).then(msg => console.log("Status: ${res.status} | Resposta: ${msg}"));
< Promise {<pending>}
Status: 201 | Resposta: Usuário criado! VM442:9
fetch('http://localhost:7070/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: "aluno_fatec",
    password: "segredo"
  })
})
.then(res => res.text()).then(msg => console.log("LOGIN: ${res.status === 200 ? 'SUCESSO' : 'FALHA'} | Msg: ${msg}"));
< Promise {<pending>}
LOGIN: SUCESSO | Msg: Login com sucesso! VM446:9
fetch('http://localhost:7070/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: "aluno_fatec",
    password: "senha_errada"
  })
})
.then(res => res.text()).then(msg => console.log("LOGIN: ${res.status === 200 ? 'SUCESSO' : 'BLOQUEADO'} | Status: ${res.status}"));
< Promise {<pending>}
POST http://localhost:7070/login 401 (Unauthorized) VM458:1
LOGIN: BLOQUEADO | Status: 401 VM458:9
fetch('http://localhost:7070/users/aluno_fatec', {
  method: 'DELETE'
})
.then(res => {
  if(res.status === 204) console.log("Usuário excluído com sucesso (Sem conteúdo de retorno)");
  else console.log("Erro ao excluir: " + res.status);
});
< Promise {<pending>}
Usuário excluído com sucesso (Sem conteúdo de retorno) VM472:5
fetch('http://localhost:7070/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: "aluno_fatec",
    password: "segredo"
  })
})
.then(res => console.log("Status pós-exclusão: ${res.status} (Esperado: 401)"));
< Promise {<pending>}
POST http://localhost:7070/login 401 (Unauthorized) VM476:1
Status pós-exclusão: 401 (Esperado: 401) VM476:9
```

Console AI assistance What's new Issues

O que é HttpSession? (Stateful)

Até agora, nossa API era **Stateless** (sem estado): a cada requisição (**POST**, **GET**), o servidor esquecia quem éramos. Para proteger rotas, precisamos que o servidor **lembre** do cliente.

- **Como funciona:**

1. O cliente faz Login.
2. O servidor cria um espaço na memória RAM (Sessão) e gera um ID único (**JSESSIONID**).
3. O servidor devolve esse ID em um **Cookie** para o navegador.
4. O navegador envia esse Cookie automaticamente em todas as próximas requisições.

- **O Javalin:** Facilita isso com `ctx.sessionAttribute()`.

Implementando Login com Sessão

Vamos alterar a rota `/login` e criar a rota `/logout` no `Main.java`. Agora, se a senha estiver correta, buscamos o usuário completo e o salvamos na sessão.

OPEN EDITORS

UserRepository.java database\src\m... 1

User.java database\src\main\java\br\gov\...

X Main.java database\src\main\java\br\gov\...

JAVADATABASE

database

src

main

java\br\gov\sp\fatcc\pg

database

SQLiteConnection.java

model

User.java

repository

UserRepository.java 1

Main.java

resources

test

target

pom.xml

users.db

UserRepository.java 1

User.java

Main.java X

database > src > main > java > br > gov > sp > fatcc > pg > Main.java > Main > main(String[])

```
9 public class Main {
10     public static void main(String[] args) {

22         // 3. Cadastrar Usuário (POST)
23         app.post("/users", ctx -> { ...

32
33         // 4. Deletar Usuário (DELETE)
34         app.delete("/users/{username}", ctx -> { ...

42
43         // 5. Login (POST) - Simples validação
44         app.post("/login", ctx -> {
45             User dto = ctx.bodyAsClass(User.class); // Dados do formulário
46             // 1. Valida credenciais
47             if (UserRepository.validate(dto.getUsername(), dto.getPassword())) {
48                 // 2. Busca o usuário completo (Aqui usamos o getByUsername!)
49                 User userDoBanco = UserRepository.getByUsername(dto.getUsername());
50                 // 3. Grava na Sessão (Javalin cria o Cookie JSESSIONID automaticamente)
51                 ctx.sessionAttribute("usuario_logado", userDoBanco);
52                 ctx.status(200).result("Login efetuado! Sessão criada.");
53             } else {
54                 ctx.status(401).result("Dados incorretos.");
55             }
56         });

57
58         // Rota de Logout (Destrói a sessão)
59         app.post("/logout", ctx -> {
60             ctx.req().getSession().invalidate(); // Limpa a memória do servidor
61             ctx.status(200).result("Sessão encerrada.");
62         });
63
64     }
```

Protegendo Rotas (Middleware)

Agora vamos criar uma regra de segurança: **"Ninguém acessa nada que comece com `/admin/` se não estiver logado"**.

No Javalin, usamos o método `.before()` para interceptar requisições.

UserRepository.java 1 User.java Main.java

```
database > src > main > java > br > gov > sp > fatcc > pg > J Main.java > Main > main(String[])
9   public class Main {
10      public static void main(String[] args) {
11
12
13
14      Javalin app = Javalin.create().start(7070);
15
16      // Antes das rotas ou logo após a criação do app
17      app.before("/admin/*", ctx -> {
18          // Verifica se existe um usuário guardado na sessão
19          User usuario = ctx.sessionAttribute("usuario_logado");
20
21          if (usuario == null) {
22              // Se não houver usuário, interrompe a requisição aqui mesmo!
23              ctx.status(401).result("Acesso negado: Faça login primeiro.");
24              ctx.skipRemainingHandlers(); // Não deixa passar para a rota real
25          }
26          // Se não entrar no if, o Javalin deixa a requisição seguir normalmente
27      });
28
29      // 2. Rota de Teste (Logger)
30      app.get("/logger", ctx -> { ...
31
32
33
34      // 3. Cadastrar Usuário (POST)
35      app.post("/users", ctx -> { ...
36
37
38
39
40
41
42
43
44      // 4. Deletar Usuário (DELETE)
45      app.delete("/users/{username}", ctx -> { ...
46
47
48
49
50
51
52
53
54      // 5. Login (POST) - Simples validação
55      app.post("/login", ctx -> { ...
56
57
58
59
60
61
62
63      // Rota de Logout (Destroi a sessão)
64      app.post("/logout", ctx -> { ...
65
66
67
68
69      // Uma rota protegida para testar
70      app.get("/admin/dados", ctx -> {
71          User usuario = ctx.sessionAttribute("usuario_logado");
72          ctx.result("Dados secretos acessados por: " + usuario.getUsername());
73      });
74
75
76
77
78
79
80
81
82
83 }
```

Testando Sessão no Chrome

O `fetch` do navegador não envia cookies em requisições cruzadas por padrão, mas como estamos no `localhost` (mesmo domínio), vai funcionar nativamente.

Roteiro de Teste (Console F12):

1. **Tentar acessar a área restrita (Sem login)**
2. **Fazer Login (Criar a Sessão)** *Certifique-se que o usuário `aluno_fatec` existe (se deletou na aula anterior, crie de novo).*
3. **Tentar acessar a área restrita (Logado)**
4. **Logout**

Conceito Importante: A `HttpSession` funciona muito bem para aplicações monolíticas (onde o front-end e o back-end são servidos pelo mesmo lugar) ou APIs simples.

Acesso negado: Faça login primeiro.

Elements Console Sources Network Performance Memory Application Privacy and security

top https://www.gstatic.com/_mss/boq-chrome-webstore/_js/k=boq-chrome-webstore.Chrom... Custom levels No Issues

7 messages
5 user messages
2 errors
No warnings
5 info
No verbose

```
GET http://localhost:7070/admin/dados 401 (Unauthorized) dados:1
fetch('http://localhost:7070/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: "aluno_fatec",
    password: "segredo"
  })
})
.then(res => res.text()).then(msg => console.log('Status: ${res.status} | Resposta: ${msg}'));
Promise {<pending>}
Status: 201 | Resposta: Usuário criado! VM824:9
fetch('http://localhost:7070/admin/dados')
.then(res => res.text()).then(msg => console.log("Tentativa 1:", msg));
Promise {<pending>}
GET http://localhost:7070/admin/dados 401 (Unauthorized) VM828:1
Tentativa 1: Acesso negado: Faça login primeiro. VM828:2
fetch('http://localhost:7070/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username: "aluno_fatec", password: "segredo" })
})
.then(res => res.text()).then(msg => console.log("Login:", msg));
Promise {<pending>}
Login: login efetuado! Sessão criada. VM832:6
fetch('http://localhost:7070/admin/dados')
.then(res => res.text()).then(msg => console.log("Tentativa 2:", msg));
Promise {<pending>}
Tentativa 2: Dados secretos acessados por: aluno_fatec VM836:2
fetch('http://localhost:7070/logout', { method: 'POST' })
.then(res => res.text()).then(msg => console.log("Logout:", msg));
Promise {<pending>}
Logout: Sessão encerrada. VM840:2
```

O Limite da Sessão e a Solução Stateless

O Problema da Escala:

- Se o servidor cair ou reiniciar, todas as sessões (**HttpSession**) são perdidas.
- Se tivermos 2 servidores (Load Balancer), a sessão criada no Servidor A não existe no Servidor B.
- Aplicativos Móveis (Android/iOS) não lidam bem com Cookies de sessão.

A Solução: Tokens (Stateless):

- O servidor não guarda o estado do usuário na memória RAM.
- O servidor gera um **Token** (um código longo e único) e entrega ao cliente.
- O cliente deve enviar esse Token no **Cabeçalho (Header)** de *todas* as requisições.
- Se o Token for válido, o servidor permite o acesso.

Evoluindo o Banco (ALTER TABLE)

Não vamos apagar o banco. Vamos alterá-lo. No método `createTable` da classe `SQLiteConnection`, adicionaremos uma tentativa de criar a coluna `token`.

Arquivo:

`src/main/java/br/gov/sp/fatec/pg/database/SQLiteConnection.java`

EXPLORER

...

OPEN EDITORS

database\src\main\... 1

database\src\main\java\br\gov\... 1

database\src\mai... 1

JAVADATABASE

database

src

main

java\br\gov\sp\fatcc\pg

database

SQLiteConnection.java

model

User.java

repository

UserRepository.java

Main.java

resources

test

target

pom.xml

users.db

UserRepository.java 1

Main.java

SQLiteConnection.java X

database > src > main > java > br > gov > sp > fatcc > pg > database > SQLiteConnection.java > SQLiteConnection > createTable()

```
6
7 public class SQLiteConnection {
8     // String de conexão (cria o arquivo users.db na raiz do projeto)
9     private static final String URL = "jdbc:sqlite:users.db";
10
11     public static Connection connect() throws Exception {
12         return DriverManager.getConnection(URL);
13     }
14
15     // Método para criar a tabela se ela não existir
16     public static void createTable() {
17         // 1. Cria a tabela base (se não existir)
18         String sqlCreate = "CREATE TABLE IF NOT EXISTS users (" +
19             "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
20             "username TEXT UNIQUE NOT NULL, " +
21             "password TEXT NOT NULL)";
22         // 2. Tenta adicionar a coluna token (Migração)
23         String sqlAlter = "ALTER TABLE users ADD COLUMN token TEXT";
24
25         // Try-with-resources garante o fechamento da conexão
26         try (Connection conn = connect();
27             Statement stmt = conn.createStatement()) {
28             stmt.execute(sqlCreate);
29             System.out.println(x: "Tabela 'users' verificada/criada com sucesso.");
30             try {
31                 stmt.execute(sqlAlter); // Tenta criar a coluna
32                 System.out.println(x: "Coluna 'token' adicionada com sucesso.");
33             } catch (Exception e) {
34                 // Se der erro, provavelmente a coluna já existe. Ignoramos.
35                 // Em produção, usaríamos ferramentas como Flyway ou Liquibase.
36                 System.out.println(x: "Nota: A coluna 'token' provavelmente já existe.");
37             }
38         } catch (Exception e) {
39             e.printStackTrace();
40         }
41     }
42 }
```

EXPLORER

OPEN EDITORS

- UserRepository.java database\src\m... 1
- ✕ Main.java database\src\main\java\br\gov\...
- SQLiteConnection.java database\src\mai...

JAVADATABASE

- database
 - src
 - main
 - java\br\gov\sp\fatec\pg
 - database
 - SQLiteConnection.java
 - model
 - User.java
 - repository
 - UserRepository.java 1
 - Main.java

resources

test

target

pom.xml

UserRepository.java 1

Main.java ✕

SQLiteConnection.java

database > src > main > java > br > gov > sp > fatec > pg > Main.java > Main > main(String[])

```
1 package br.gov.sp.fatec.pg;
2
3 import br.gov.sp.fatec.pg.database.SQLiteConnection;
4 import io.javalin.Javalin;
5
6 import br.gov.sp.fatec.pg.repository.UserRepository;
7 import br.gov.sp.fatec.pg.model.User;
8
9 public class Main {
10     public static void main(String[] args) {
11         // 1. Garante que a tabela existe ao iniciar
12         SQLiteConnection.createTable();
13     }
14 }
```

Run | Debug

OUTPUT

DEBUG CONSOLE

PORTS

TERMINAL

Run: Main + -

PS C:\Users\rlarg\OneDrive\Documentos\Java\JavaDatabase> & 'C:\Program Files\Java\openjdk-19.0.2\bin\java.exe' '@C:\Users\rlarg\AppData\Local\Temp\cp_93dvazg3ih4cp8cr8j03hvvrd.argfile' 'br.gov.sp.fatec.pg.Main'

Tabela 'users' verificada/criada com sucesso.
Coluna 'token' adicionada com sucesso.

[main] INFO io.javalin.Javalin - Starting Javalin ...

[main] INFO org.eclipse.jetty.server.Server - jetty-11.0.25; built: 2025-03-13T00:15:57.301Z; git

Atualizando o Repositório

Precisamos de métodos para salvar o token no banco (ao logar) e buscar o usuário pelo token (ao validar a requisição).

Arquivo: `src/main/java/br/gov/sp/fatec/pg/repository/UserRepository.java`

EXPLORER

OPEN EDITORS 1 unsaved

UserRepository.java database/src/main... 1

Main.java database/src/main/java/br/gov/...

SQLiteConnection.java database/src/main/...

JAVADATABASE

database

src

main

java\br\gov\sp\fatcc\pg

database

SQLiteConnection.java

model

User.java

repository

UserRepository.java 1

Main.java

resources

test

target

pom.xml

users.db

UserRepository.java 1 • J Main.java J SQLiteConnection.java

database > src > main > java > br > gov > sp > fatcc > pg > repository > J UserRepository.java > UserRepository

11 public class UserRepository {

38 > public static boolean delete(String username) throws Exception { ...

50

51 > /** ...

58 > public static boolean validate(String username, String password) throws Exception { ...

75

76 > /** ...

79 > public static User getByUsername(String username) throws Exception { ...

98

99

// ATUALIZAR TOKEN (Login)

100 public static void updateToken(String username, String token) throws Exception {

101 String sql = "UPDATE users SET token = ? WHERE username = ?";

102 try (Connection conn = SQLiteConnection.connect();

103 PreparedStatement pstmt = conn.prepareStatement(sql)) {

104 pstmt.setString(parameterIndex: 1, token);

105 pstmt.setString(parameterIndex: 2, username);

106 pstmt.executeUpdate();

107 }

108 }

109

// BUSCAR POR TOKEN (Validação de Acesso)

111 public static User getUserByToken(String token) throws Exception {

112 String sql = "SELECT * FROM users WHERE token = ?";

113 try (Connection conn = SQLiteConnection.connect();

114 PreparedStatement pstmt = conn.prepareStatement(sql)) {

115 pstmt.setString(parameterIndex: 1, token);

116 ResultSet rs = pstmt.executeQuery();

117 if (rs.next()) {

118 User u = new User();

119 u.setUsername(rs.getString(columnLabel: "username"));

120 // Não precisamos retornar a senha aqui por segurança

121 return u;

122 }

123 }

124 return null; // Token inválido ou não encontrado

125 }

126 }

A Nova Rota: “/token-login”

No `Main.java`, vamos manter a rota antiga e adicionar a nova. Observe a diferença de comportamento.

EXPLORER

... J UserRepository.java 1 • J Main.java • J SQLiteConnection.java

OPEN EDITORS 2 unsaved

- J UserRepository.java database\src\m... 1
- J Main.java database\src\main\java\br\gov\... 1
- J SQLiteConnection.java database\src\mai...

JAVADATABASE

- database
 - src
 - main
 - java\br\gov\sp\fatcc\pg
 - database
 - SQLiteConnection.java
 - model
 - User.java
 - repository
 - UserRepository.java 1
 - Main.java
 - resources
 - test
 - target
 - pom.xml
 - users.db

```
database > src > main > java > br > gov > sp > fatcc > pg > J Main.java > Main > main(String[])
11 public class Main {
12     public static void main(String[] args) {
57
58     // 5. Login (POST) - Simples validação
59 > app.post("/login", ctx -> { ...
72
73     // 5.1. (Login com Token) - Stateless
74 app.post("/token-login", ctx -> {
75     User dto = ctx.bodyAsClass(User.class);
76
77     if (UserRepository.validate(dto.getUsername(), dto.getPassword())) {
78         // 1. Gerar um Token aleatório único (UUID)
79         String token = UUID.randomUUID().toString();
80
81         // 2. Salvar o token no banco (Persistência)
82         UserRepository.updateToken(dto.getUsername(), token);
83
84         // 3. Retornar o token para o cliente (JSON)
85         ctx.json(java.util.Map.of("token", token));
86     } else {
87         ctx.status(401).result("Credenciais inválidas");
88     }
89 }
90
91 // Rota de Logout (Destroi a sessão)
92 > app.post("/logout", ctx -> { ...
96
97 // Uma rota protegida para testar
98 > app.get("/admin/dados", ctx -> { ...
102
103 }
```

Protegendo uma Área de API (Middleware)

Vamos definir que qualquer rota começando com `/api/*` será protegida por **Token**, enquanto `/admin/*` continua protegida por **Sessão**.

Arquivo: `Main.java`

EXPLORER

OPEN EDITORS 2 unsaved

- UserRepository.java database\src\m... 1
- Main.java database\src\main\java\br\gov\... 1
- SQLiteConnection.java database\src\mai...

JAVADATABASE

- database
 - src
 - main
 - java\br\gov\sp\fatcc\pg
 - database
 - SQLiteConnection.java
 - model
 - User.java
 - repository
 - UserRepository.java 1
 - Main.java
 - resources
 - test
 - target
 - pom.xml
 - users.db

UserRepository.java 1 Main.java 1 SQLiteConnection.java

database > src > main > java > br > gov > sp > fatcc > pg > Main.java > Main > main(String[])

```
11 public class Main {
12     public static void main(String[] args) {
13
14
15
16         Javalin app = Javalin.create().start(7070);
17
18         // Antes das rotas ou logo após a criação do app
19         app.before("/admin/*", ctx -> { ...
20
21
22
23
24
25
26
27
28
29
30
31         // Middleware para proteger a API via TOKEN
32         app.before("/api/*", ctx -> {
33             // 1. Pega o token do cabeçalho (Padrão: "Authorization")
34             String token = ctx.header("Authorization");
35
36             if (token == null) {
37                 ctx.status(401).result("Cabeçalho Authorization ausente.");
38                 ctx.skipRemainingHandlers(); // Para tudo!
39                 return;
40             }
41
42             // 2. Verifica se o Token existe no Banco
43             User usuario = UserRepository.getUserByToken(token);
44
45             if (usuario == null) {
46                 ctx.status(403).result("Token inválido ou expirado. Faça login novamente.");
47                 ctx.skipRemainingHandlers();
48             } else {
49                 // Token válido! Deixa passar.
50                 System.out.println("Acesso API autorizado para: " + usuario.getUsername());
51             }
52         });
53
54         // Exemplo de rota protegida por Token
55         app.get("/api/secreto", ctx -> {
56             ctx.result("Se você está vendo isso, seu Token é válido!");
57         });
58     }
59 }
```

Testando o Fluxo Stateless (Chrome)

Vamos testar exclusivamente o fluxo novo via Console (F12).

- 1. Obter o Token (Login)**
- 2. Tentar Acesso sem Token (Deve falhar)**
 - a. Esperado: 401 Unauthorized*
- 3. Acesso com Token (Deve funcionar)**
 - a. Esperado: Com token: Se você está vendo isso, seu Token é válido!*

Cabeçalho Authorization ausente.

Elements Console Sources Network Performance Memory Application Privacy and security

top

5 messages

3 user messages

2 errors

No warnings

3 info

No verbose

```
GET http://localhost:7070/api/secreto 401 (Unauthorized) secreto:1
```

```
var myToken = "";
```

```
fetch('http://localhost:7070/token-login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username: "aluno_fatec", password: "segredo" })
})
.then(res => res.json())
.then(data => {
  console.log("Token Recebido:", data.token);
  myToken = data.token; // Guardamos na variável para usar depois
});
```

Promise {<pending>}

Token Recebido: 82343cd4-4e7a-486c-8869-88cbbf4b7a76 VM921:10

```
fetch('http://localhost:7070/api/secreto')
.then(r => console.log("Sem token:", r.status, r.statusText));
```

Promise {<pending>}

```
GET http://localhost:7070/api/secreto 401 (Unauthorized) VM925:1
```

Sem token: 401 Unauthorized VM925:2

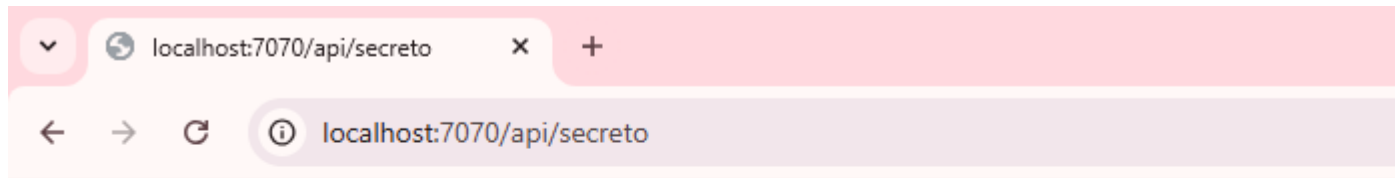
```
fetch('http://localhost:7070/api/secreto', {
  headers: { 'Authorization': myToken } // Enviando o UUID no header
})
.then(r => r.text())
.then(msg => console.log("Com token:", msg));
```

Promise {<pending>}

Com token: Se você está vendo isso, seu Token é válido! VM929:5

Corrigindo erros de codificação de caracteres

Esse problema acontece quando o servidor envia o texto codificado em **UTF-8** (onde 'ç' são dois bytes), mas o navegador ou cliente (Postman, etc.) assume que é **ISO-8859-1** (Latin-1) e interpreta cada byte separadamente.



Cabeçalho `Authorization` ausente.

Solução

EXPLORER

OPEN EDITORS

- UserRepository.java database\src\m... 1
- Main.java database\src\main\java\br\gov\... 1
- SQLiteConnection.java database\src\mai...

JAVADATABASE

- database
 - src
 - main
 - java\br\gov\sp\fatcc\pg
 - database
 - SQLiteConnection.java
 - model
 - User.java
 - repository
 - UserRepository.java 1
 - Main.java- resources
- test
- target
- pom.xml
- users.db

UserRepository.java 1

Main.java X

SQLite

database > src > main > java > br > gov > sp > fatec > pg >

```
4
5 import br.gov.sp.fatec.pg.database.SQLiteConnection;
6 import io.javalin.Javalin;
7
8 import br.gov.sp.fatec.pg.repository.UserRepository;
9 import br.gov.sp.fatec.pg.model.User;
10
11 public class Main {
12     Run | Debug
13     public static void main(String[] args) {
14         // 1. Garante que a tabela existe ao iniciar
15         SQLiteConnection.createTable();
16
17         Javalin app = Javalin.create(config -> {
18             // Define o Content-Type padrão para todas as respostas
19             config.http.defaultContentType = "text/plain; charset=UTF-8";
20
21             // Se estiver usando arquivos estáticos ou templates, verifique
22             config.pvt.javaLangErrorHandler((res, error) -> {
23                 res.setCharacterEncoding("UTF-8");
24             });
25         }).start(7070);
```

Cabeçalho Authorization ausente.

